Documentation Technique — Séquences API HBnB

Introduction

1. Ce document présente la logique de flux des appels API pour l'application HBnB.
   Il se concentre sur 3 cas d'utilisation principaux :

1. *Création d'un utilisateur (User Registration)*

2. *Envoie d'un avis (Review submission)*

3. *Récupération de la liste des logements (Place Listing)*

4. L'architecture générale

5. La conception détaillée de la couche Business Logic

6. Les flux d'interaction des appels API

Architecture Générale

L'objectif est de montrer ***comment les différentes couches interagissent*** *:*

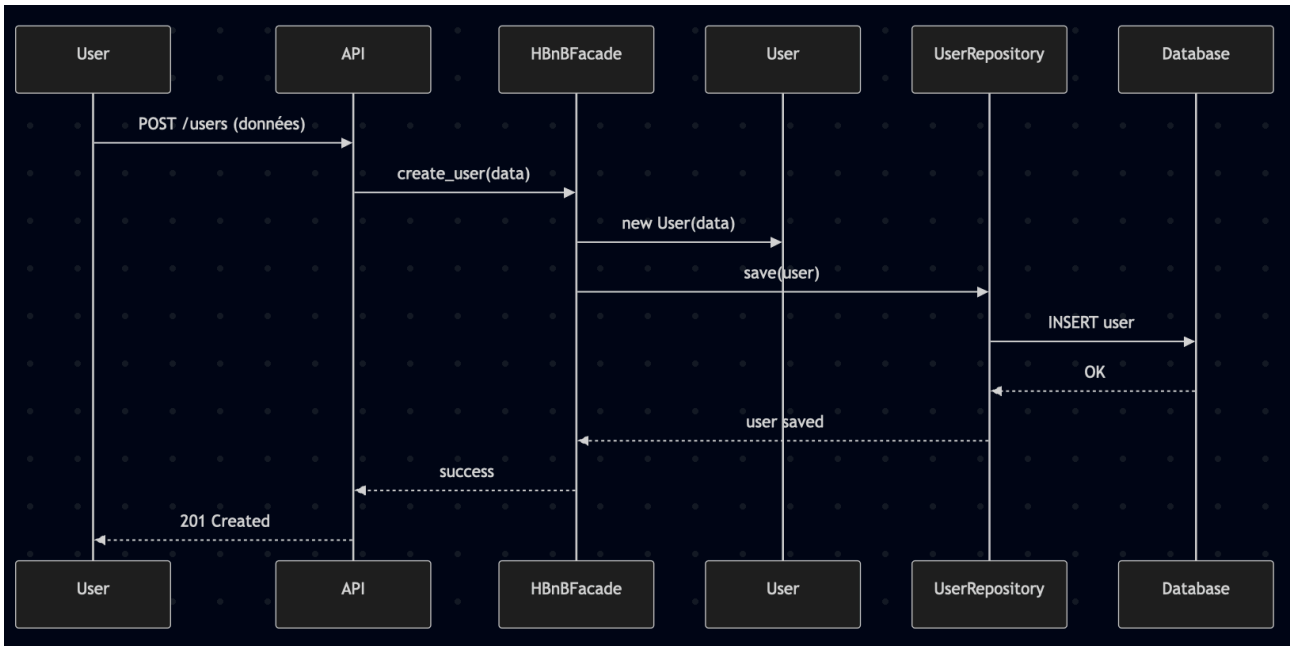
Présentation (API/Services), Logique Métier (Business Logic), et Persistence (Database).

Ces diagrammes servent de référence pour l'implémentation et la compréhension des flux de l'application.

User Registration (Inscription utilisateur)

Objectif

Permettre à un utilisateur de créer un compte HBnB.



<u>Explication</u>

- *L'API reçoit la requête et délègue à la **Facade**, qui applique la logique métier.*
- *La **UserRepository** gère l'insertion en base de données.*
- *La réponse retourne **201 Created** si tout est valide.*

Review Submission (Soumission d'un avis)

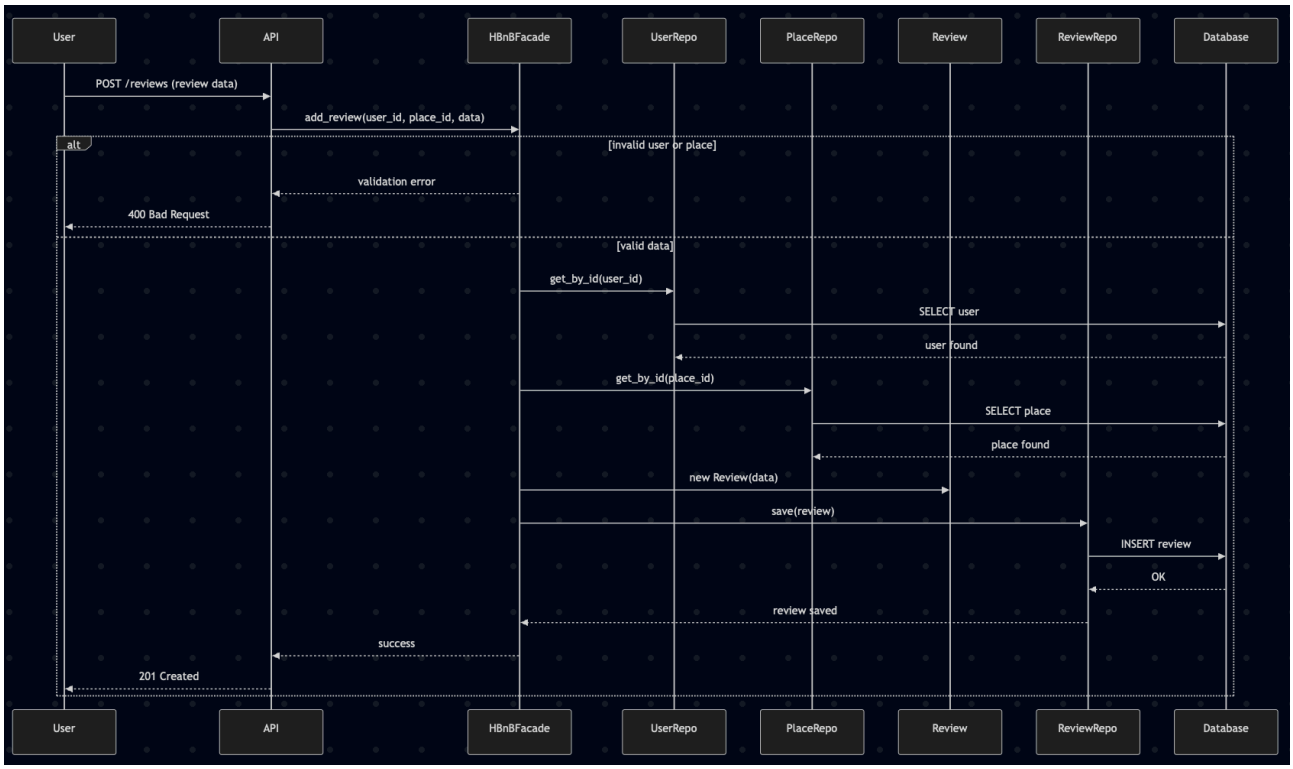
Objectif

<u>Permettre à un utilisateur de **soumettre un avis** sur un logement, en validant :</u>

- *L'utilisateur existe*

- *Le logement existe*
- *Les données de l'avis sont valides (note, commentaire)*

Si une validation échoue, une erreur est retournée.

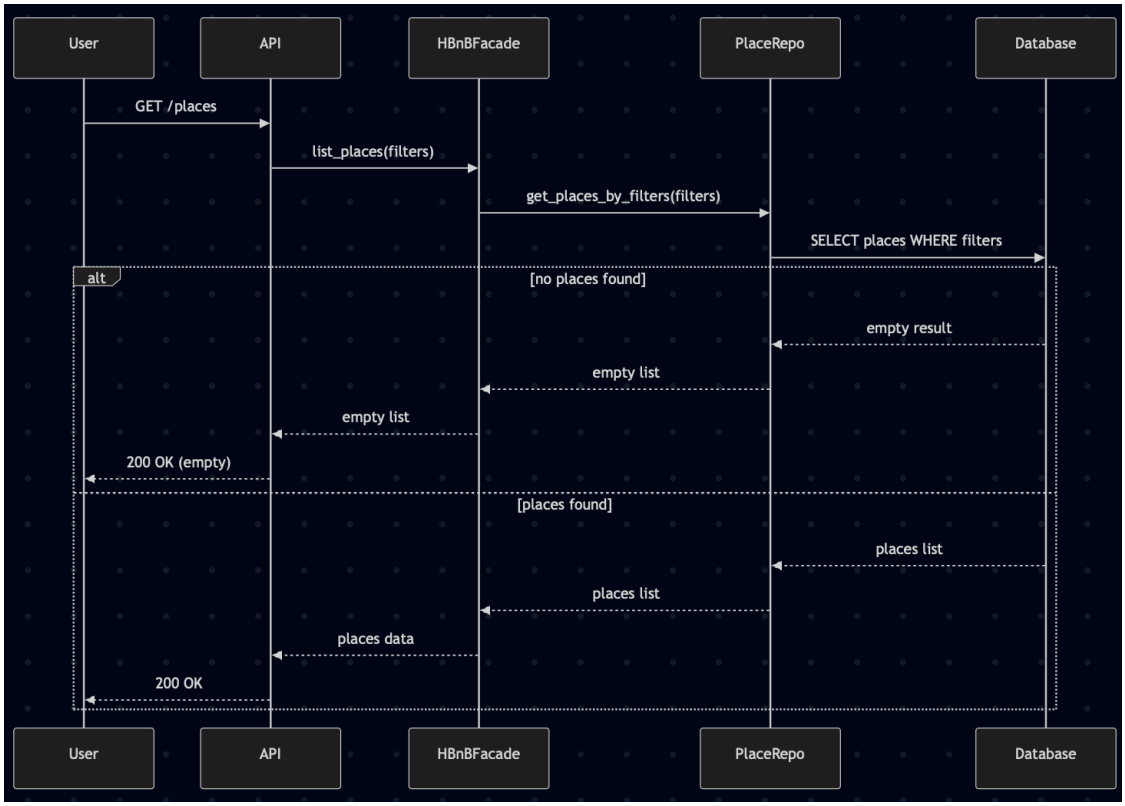


Explication

1. ***API** reçoit la requête POST /reviews avec les données de l'avis.*

2. ***La Facade add_review() valide :***
   - *L'utilisateur existe*
   - *Le logement existe*
   - *Les données de l'avis sont correctes (validate_rating() par exemple)*
3. ***Cas d'erreur** : Si l'utilisateur ou le logement n'existe pas, la Facade retourne une erreur → 400 Bad Request.*

4. ***Cas succès** :*



- *L'utilisateur et le logement sont récupérés via les repositories*
- *Un nouvel objet Review est créé*
- *La Review est sauvegardée dans la base via ReviewRepository*

5. *L'API renvoie **201 Created** avec l'objet nouvellement créé.*

Objectif

Permettre à un utilisateur de ***parcourir les logements disponibles***, avec gestion du cas où ***aucun logement n'est trouvé***.

Ce diagramme montre comment ***les différentes couches interagissent*** pour récupérer et renvoyer les données.

Explication

1. ***L'utilisateur*** fait une requête GET /places, éventuellement avec des filtres.

2. ***L'API*** transmet la requête à la **Facade** (list_places) pour centraliser la logique métier.

3. La **Facade** demande à PlaceRepository d'interroger la base de données selon les filtres.

4. ***<u>La Base de données renvoie soit :</u>***
   - Une **liste vide** → réponse 200 OK (empty)
   - Une **liste de logements** → réponse 200 OK avec les données

5. L'API renvoie les résultats à l'utilisateur.

Notes

- *La **Facade** centralise la logique métier, même si aucune création d'objet complexe n'est nécessaire.*
- *Les **repositories** gèrent uniquement l'accès aux données.*
- Ce flux garantit une **séparation claire des responsabilités** :
  - *L'API gère la requête et la réponse HTTP*
  - *La logique métier filtre et organise les données*
  - *La persistence interagit avec la base de données*

Architecture Générale

Vue d'ensemble

HBnB suit une architecture en trois couches :

1. Couche Présentation
2. Couche Business Logic
3. Couche Persistance

Un pattern **Facade** est utilisé entre la couche Présentation et la Business Logic afin de simplifier les interactions.

Description des couches

Couche Présentation

Contient :

- *L'API*
- *Les services exposés*

Responsabilités :

- *Recevoir les requêtes HTTP*
- *Vérifier le format des données*
- *Retourner les réponses HTTP*
- *Transmettre les demandes à la Business Logic*

Cette couche ne contient aucune règle métier.

Couche Business Logic

Contient les entités principales :

- User
- Place
- Review
- Amenity

Responsabilités :

- Appliquer les règles métier
- Valider les données fonctionnelles

- Gérer les relations entre entités
- Coordonner l’accès aux données

C’est le cœur du système.

Couche Persistance

Contient :

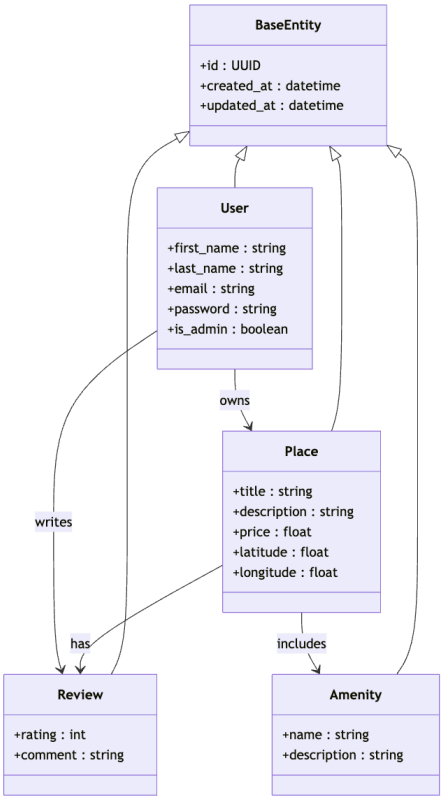
- Les repositories
- La base de données

Responsabilités :

- Stocker les données
- Récupérer les données
- Abstraire la logique d’accès à la base

La Business Logic n’accède jamais directement à la base.

Couche Business Logic



Vue d’ensemble

La couche Business Logic définit les entités principales du domaine ainsi que leurs relations.

Toutes les entités héritent d’une classe commune : **BaseEntity**.

Description des entités

BaseEntity

Classe mère commune.

Attributs :

- id (UUID)
- created_at
- updated_at

Permet une identification unique et un suivi temporel.

User

Représente un utilisateur du système.

Attributs :

- first_name
- last_name
- email
- password
- is_admin

Règles métier :

- Email unique
- Mot de passe sécurisé
- Gestion des privilèges administrateur

Place

Représente un logement.

Attributs :

- title
- description
- price
- latitude
- longitude

Règles métier :

- Prix positif
- Coordonnées valides

Review

Représente un avis.

Attributs :

- rating
- comment

Règles métier :

- Rating dans une plage définie
- Respect des contraintes d’association utilisateur/place

Amenity

Représente un équipement disponible.

Attributs :

- name
- description

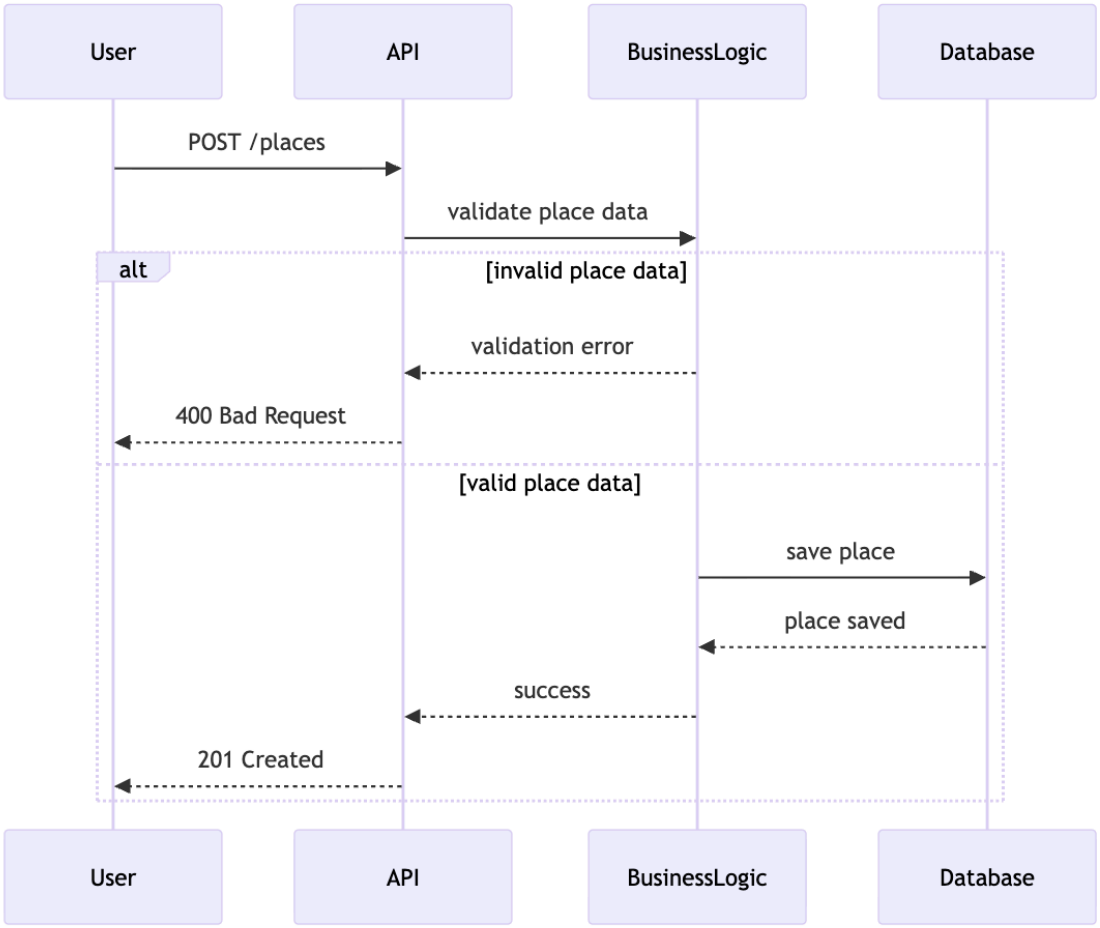
Relations entre entités

- Un User possède des Places
- Un User écrit des Reviews
- Une Place contient des Reviews

- Une Place inclut des Amenities

Ces relations traduisent directement les règles fonctionnelles du système.

Flux d'Interaction API



Objectif

Cette section décrit le fonctionnement interne lors de la création d'un nouveau logement (Place).

Déroulement du processus

Étape 1 : Requête

L'utilisateur envoie une requête POST pour créer un nouveau logement.

L'API reçoit la requête.

Étape 2 : Validation

L’API transmet les données à la Business Logic.

La Business Logic :

- Vérifie les champs
- Applique les règles métier
- Contrôle les contraintes

Cas 1 : Données invalides

Si les données ne respectent pas les règles :

- Une erreur est générée
- L’API retourne 400 Bad Request

Cas 2 : Données valides

Si les données sont valides :

- La Business Logic demande l’enregistrement
- La couche Persistance sauvegarde les données
- L’API retourne 201 Created

Principes d’Architecture

Principes appliqués

- Séparation stricte des responsabilités
- Aucune logique métier dans l’API
- Pas d’accès direct à la base depuis la Présentation
- Héritage commun via BaseEntity

Avantages

<u>Cette architecture garantit :</u>

- Une meilleure maintenabilité
- Une meilleure évolutivité

- Une organisation claire
- Une facilité de test
- Une meilleure compréhension globale

Conclusion

Ce document regroupe :

- L’architecture générale du système
- La conception détaillée de la couche Business Logic
- Le flux d’interaction des appels API

Il constitue une base technique solide pour l’implémentation du projet HBnB Evolution et servira de référence tout au long du développement.